Fuzhao Xue, Valerii Likhosherstov, Anurag Arnab, Neil Houlsby, Mostafa Dehghani, and Yang You. Adaptive computation with elastic input sequence. In *International Conference on Machine Learning, ICML 2023*, Proceedings of Machine Learning Research. PMLR, 2023.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

Zexuan Zhong, Dan Friedman, and Danqi Chen. Factual probing is [MASK]: Learning vs. learning to recall. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023*. OpenReview.net, 2023.

A PRELIMINARIES: TRANSFORMER

Consider a vocabulary $\mathcal{V}$ and an input $\mathbf{p}_{1:K} \in \mathcal{V}^K$ of K tokens, and an L-layer decoder-only language model. The l'th layer of the Transformer produces one intermediate vector for each token here, denoted by $\mathbf{v}_k^{(l)} \in \mathbb{R}^D$ for $k = 1, \ldots, K$. We first describe this operation before outlining the end-to-end next-token generation process.

Consider a Transformer (Vaswani et al., 2017) block $T(\cdot) : \mathbb{R}^{K \times D} \to \mathbb{R}^{K \times D}$ that operates over a sequence of K intermediate vectors. The block is defined by H many sets of four matrices each, $W_{\text{query}}^{(h)}, W_{\text{key}}^{(h)}, W_{\text{value}}^{(h)}$ and $W_{\text{out}}^{(h)}, \in \mathbb{R}^{D_{\text{attn}} \times D}$ (for $h = 1, \ldots H$ each denoting an attention head), and a single parameterized feedforward module $f_{\text{FF}} : \mathbb{R}^D \to \mathbb{R}^D$. Let $\Phi_{\text{LN}} : \mathbb{R}^D \to \mathbb{R}^D$ denote the layer-norm operation. Given the input vectors $V_{1:K} \in \mathbb{R}^{D \times K}$, the output $V'_{1:K}$ of the Transformer block $T(\cdot)$ can be expressed in the following steps. For all $k \leq K$,

$$\mathbf{a}_k = \Phi_{\text{LN}}^{(1)} \left(\mathbf{v}_k + \sum_{h=1}^{H} (W_{\text{out}}^{(h)})^T \cdot W_{\text{value}}^{(h)} V_{1:k} \text{softmax} \left(\frac{(W_{\text{key}}^{(h)} V_{1:k})^T W_{\text{query}}^{(h)} \mathbf{v}_k}{\sqrt{D_{\text{attn}}}} \right) \right) \tag{4}$$

$$\mathbf{v}'_k = \Phi_{\text{LN}}^{(2)} \left(\Phi_{\text{FF}}(\mathbf{a}_k) + \mathbf{a}_k \right). \tag{5}$$

Here, the first step computes K different self-attention outputs by attending to all K input vectors, while the second step individually processes each attention output via a feedforward network and other normalization components to produce the final output of the block. Note that here we have assumed a unidirectional attention mechanism; for a bidirectional mechanism over the whole K-length prefix, one simply needs to replace $V_{1:k}$ with $V_{1:K}$ in the above computation.

Given this block, the Transformer generates the next token as follows. Let $\Phi_{\text{token}} : \mathcal{V} \to \mathbb{R}^D$ and $\Phi_{\text{position}} : \mathbb{N} \to \mathbb{R}^D$ denote the token-embedding and position-embedding layers. With an abuse of notation, let the token unembedding layer be denoted as Φ_{token}^{-1}, which maps from $\mathbb{R}^D$ to a probability vector in $\Delta^{|\mathcal{V}|-1}$. Let $T^{(l)}(\cdot)$ denote the lth Transformer layer. Then, the Transformer commits the following operations in sequence to arrive at the $(K+1)^{\text{th}}$ token.

$$\mathbf{v}_k^{(0)} = \Phi_{\text{token}}(p_k) + \Phi_{\text{position}}(k) \tag{6}$$

$$V_{1:K}^{(l)} = T^{(l)}(V_{1:K}^{(l-1)}), \forall l \in [1, L] \tag{7}$$

$$p_{K+1} \sim \Phi_{\text{token}}^{-1}(\mathbf{v}_K^{(L)}). \tag{8}$$

For a more detailed mathematical exposition of the Transformer model, we refer the reader to Thickstun (2021).

B ADDITIONAL DOWNSTREAM FINETUNING RESULTS

We first report the downstream finetuning performance for the 1B model in Table 1 (numbers corresponding to Figure 3 in §4.3). Further, in Figure 5 we report downstream performance on various tasks for a 130M decoder-only model. Again we observe that `PausePT_PauseFT` clearly outperforms standard training baseline (`StdPT_StdFT`) on GSM8k, CommonSenseQA, LAMBADA and on our fact recall tasks like WebQA and NaturalQA. However, surprisingly, we do not observe gains on SQuAD, in contrast to the gains observed in 1B model. Overall, we see an improvement in six tasks for the smaller model (one of which is PhysicalIQA where the gain is minimal).

C PREPENDING VS APPENDING PAUSE TOKENS

In Section 5, we discussed the effect of prepending the pause token in comparison to the default approach of appending them to the end of prefix. Table 2 compares the two approaches. As stated before in Section 5, for the `PausePT_PauseFT` training algorithm, we observe that prepending the pause tokens still outperforms the baseline but is (slightly) worse than appending the pause tokens on some benchmarks like GSM8k and SQuAD. For `StdPT_PauseFT` however, we see mixed results with equal number of wins and losses between the prepending and appending.

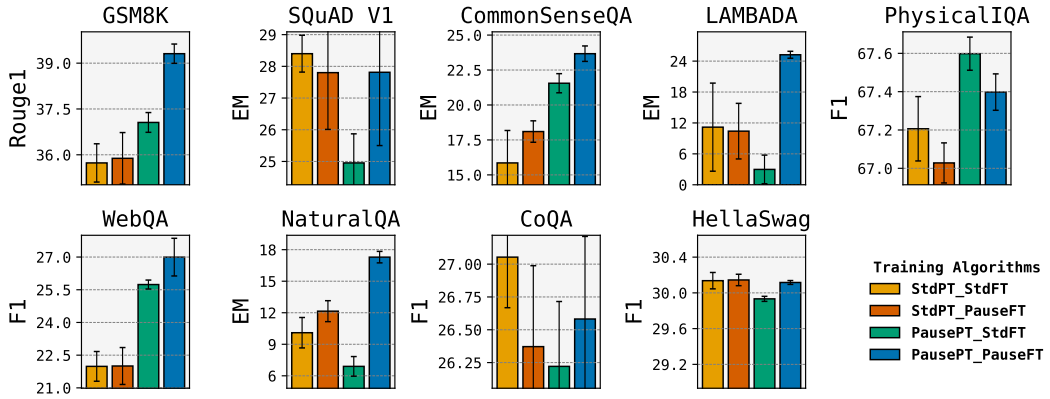

Figure 5: **Downstream performance of pause-training on a 130M decoder-only model.** We find on six out of our nine downstream tasks, the pause-pretrained and pause-finetuned model (`PausePT_PauseFT`) outperforms standard training (`StdPT_StdFT`) on a 130M decoder-only model. For example, on the reasoning task of GSM8k, we observe 3% gains in Rouge1 scores (we compare Rouge1 as the final accuracy was too low to be meaningful for our 130M model). Similarly on the general understanding task of CommonSenseQA, we observe upto 8% gains. We note here that we solve CommonsenseQA as a decoding task and not rank classification task, and hence report the Exact Match scores. We also highlight that while pause-trainined on the 1B model showed significant gains on SQuAD, they disappear here.

Dataset	**Metric**	`StdPT_StdFT`	`StdPT_PauseFT`		`PausePT_StdFT`	`PausePT_PauseFT`	
			10	**50**		**10**	**50**
GSM8k	Acc	7.5 ± 0.5	6.9 ± 1.0	6.5 ± 0.8	7.7 ± 0.5	8.5 ± 0.9	7.7 ± 0.3
	Rouge1	42.3 ± 0.5	41.7 ± 0.7	41.2 ± 1.3	43.5 ± 0.1	44.2 ± 0.2	44.1 ± 0.2
SQuAD	EM	36.4 ± 2.5	36.6 ± 2.2	40.2 ± 3.2	38.4 ± 2.9	51.7 ± 2.3	55.9 ± 1.0
CommonSense QA	EM	26.9 ± 2.9	28.8 ± 2.8	28.7 ± 2.0	27.7 ± 2.7	34.8 ± 1.2	32.3 ± 0.8
LAMBADA	EM	16.4 ± 1.7	18.4 ± 0.3	18.5 ± 0.6	13.7 ± 5.1	18.8 ± 0.1	18.5 ± 0.2
Web Questions	EM	13.7 ± 2.1	9.0 ± 4.4	12.4 ± 2.6	15.0 ± 2.5	13.8 ± 3.7	16.0 ± 1.6
Natural Questions	EM	23.6 ± 1.2	24.3 ± 1.4	23.9 ± 1.3	24.3 ± 7.5	24.9 ± 1.3	26.9 ± 0.4
CoQA	F1	29.9 ± 1.0	30.7 ± 0.5	30.3 ± 0.5	31.1 ± 0.3	31.3 ± 1.1	31.6 ± 0.5
PhysicalIQA	F1	73.3 ± 0.2	73.9 ± 0.2	74.0 ± 0.2	74.1 ± 0.2	74.1 ± 0.1	74.2 ± 0.2
HellaSwag	F1	37.8 ± 0.1	37.9 ± 0.2	37.8 ± 0.2	37.9 ± 0.1	37.7 ± 0.2	37.8 ± 0.2

Table 1: **Downstream performance on various tasks for the 1B decoder-only model.** We observe that `PausePT_PauseFT` outperforms the standard training baseline on 8 out of the 9 tasks considered in this work. See §4.3 and Figure 3 for further details.

D ZERO-SHOT EVALUATION

In §4.3, we showed the efficacy of pause-pretrained models when finetuned with pause tokens. However, we note that we witness gains even in the zero-shot setting, where the model is not finetuned. In Figure 6, we compare the zero-shot accuracy of the standard pretrained 1B model with that of pause-pretrained version. For the pause-pretrained model, we perform evaluation with 0, 10 and 50 `<pause>` tokens appended to the prefix. Observe that pause-pretraining gives some gains on tasks like GSM8k and HellSwag. However, we note here the absolute value of our zero-shot accuracies are quite low, as we experiment with a small 1B parameter model.